Attributing and Reducing End-to-End Latency in RISC-V–FPGA Neural Inference on BeagleV-Fire

Haotian TIAN
ERG2081: Independent Study I
2026.11.8

***Abstract*—Kernel-level acceleration does not guarantee faster application-level inference on an FPGA SoC, because host preprocessing, shared-memory access, and per-call driver setup can dominate a small workload. This report investigates that gap on BeagleV-Fire, whose PolarFire SoC integrates a Linux-capable RISC-V subsystem and FPGA fabric. A fixed integer MLP with unsigned 8-bit inputs and activations, signed 8-bit weights, and 32-bit accumulation ($196 \to 32 \to 10$, 6,634 parameters) is evaluated on all 10,000 MNIST test images; file I/O is excluded. Accelerator-call time is analyzed as $T_{\text{call}}(B) = H + BK$, where H is the fixed per-call cost and K is the incremental cost per image. Matched ablations vary weight placement, first-layer neuron parallelism, host staging, memory-mapped I/O (MMIO) lifetime, and batch size while requiring bit-exact agreement with the CPU reference for all 100,000 logits. The initial matched FPGA pipeline at $B = 1$ required approximately twice the latency of the single-thread CPU reference. Mapping weights into dedicated large-SRAM (LSRAM) blocks freed logic for wider parallelism without materially reducing latency; four-lane hardware then reduced K by about 35%, while persistent MMIO mapping reduced H from 121.570 to 8.159 μs/call. Combined with cacheable host staging, the final system reached 37.839 μs/image at $B = 8$, corresponding to approximately 26,428 images/s, 96.36% of the maximum measured throughput, and a 2.89× speedup over the CPU reference. At $B = 256$, it reached 36.464 μs/image, or approximately 3.00×. On this board and workload, separating fixed and per-image costs exposed the limiting subsystem and guided the final design.**

***Index Terms*—RISC-V, FPGA, hardware–software co-design, neural-network inference, SmartHLS, MNIST, BeagleV-Fire**

I. Introduction

FPGA-based neural-network accelerators are often evaluated using synthesized cycle counts or kernel-level latency. These metrics are important, but they do not represent the complete latency observed by a Linux application. Before and after an accelerator call, the host processor must preprocess input data, stage shared buffers, configure memory-mapped control registers, start the accelerator, wait for completion, and retrieve its outputs. For a small neural network, these fixed software and communication costs can be comparable to, or larger than, the arithmetic performed in the FPGA. A faster register-transfer-level (RTL) kernel can therefore produce a slower application-level system.

Frameworks such as FINN and hls4ml demonstrate that quantized neural networks can be translated into efficient FPGA data paths [2]–[4], while surveys identify memory movement and software integration as important constraints in resource-limited edge inference [5]. These works establish the feasibility of specialized FPGA inference, but they do not by themselves determine which part of a complete Linux-hosted inference path limits performance on a compact RISC-V–FPGA SoC. BeagleV-Fire provides a useful platform for this question because its PolarFire SoC combines a Linux-capable RISC-V subsystem, FPGA fabric, shared memory, and memory-mapped control interfaces on one device [11], [12].

This report asks a systems question: which components prevent a correct FPGA neural kernel from outperforming a CPU implementation, and which hardware or software intervention removes each bottleneck? The study uses a fixed $196 \to 32 \to 10$ integer MLP and the complete 10,000-image MNIST test set [10]. The model, quantization rules, and test data remain unchanged throughout the experiments, and every accepted configuration must reproduce all predictions and 100,000 logits exactly. The RISC-V CPU performs image reduction, shared-buffer staging, accelerator control, and output postprocessing, while the FPGA evaluates the two fully connected layers. Here, full-pipeline latency refers to the in-memory path from preprocessing through output postprocessing; file I/O and batch-formation waiting are excluded.

The first matched full-pipeline measurement showed why kernel timing alone was insufficient. At $B = 1$, the initial P0 pipeline required 219.608 μs/image, or 2.007 times the latency of the 109.393 μs/image single-thread CPU reference. This negative result motivated bottleneck attribution before further hardware parallelization.

The main contributions are:

- a full-pipeline measurement method that distinguishes accelerator-call latency from full-pipeline latency and analyzes call time using $T_{\text{call}}(B) = H + BK$;
- controlled hardware and software ablations showing that LSRAM mapping primarily releases FPGA logic resources, neuron-lane parallelism primarily reduces the incremental per-image cost K, persistent MMIO primarily reduces the fixed invocation cost H, and cacheable staging reduces CPU preprocessing cost; and
- a bit-exact final implementation that reaches 37.839 μs/image at $B = 8$, the first batch satisfying the predefined compute-efficiency knee criterion while retaining 96.36% of maximum measured throughput, and 36.464 μs/image at the maximum measured batch size, corresponding to 2.891× and approximately 3.00× the throughput of the CPU reference.

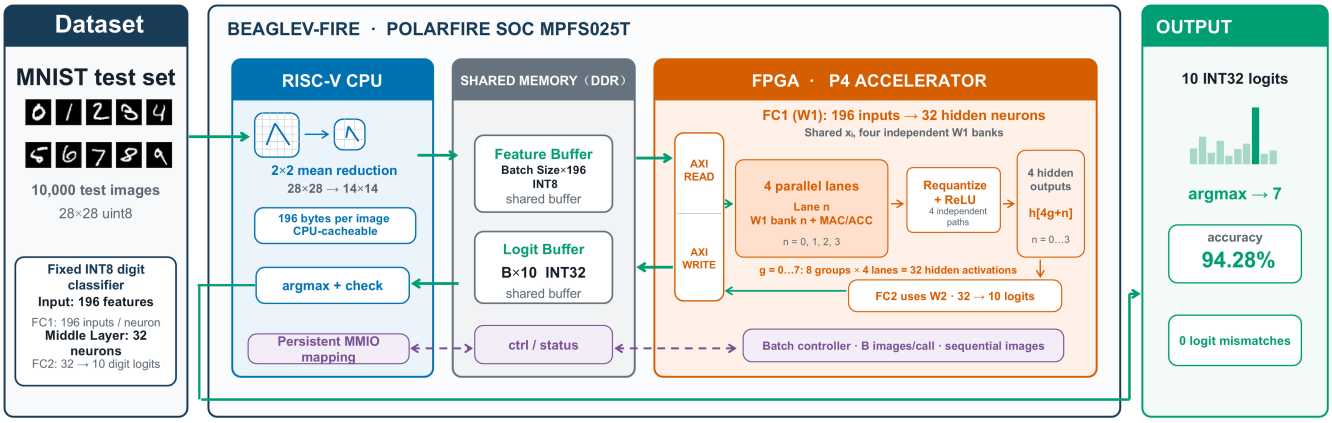


Fig. 1. Final in-memory CPU–FPGA inference path. The RISC-V CPU reduces each 28×28 MNIST image to 196 unsigned 8-bit features in cacheable memory, copies the batch contiguously into a non-cacheable shared-DDR feature buffer, and retains the MMIO mapping across calls. The P4 accelerator reads $B \times 196$ features through AXI, evaluates four FC1 hidden neurons concurrently from independent W1 banks, requantizes and applies ReLU, and evaluates FC2 to produce $B \times 10$ INT32 logits. The CPU then performs argmax; bit-exact checking is a validation step excluded from latency. Images in a batch are processed sequentially; the four-way parallelism is across hidden neurons, not across images.

The work is intended as a documented case study of bottleneck attribution on one resource-constrained RISC-V–FPGA platform. It does not propose a new neural architecture or claim that the measured speedups generalize to all networks or FPGA systems. Figure 1 summarizes the final optimized execution path.

II. Related Work and Positioning

Early FPGA inference work demonstrated low-latency quantized neural networks through aggressive quantization and customized data paths [1], [4]. hls4ml made this style of design accessible through a high-level synthesis workflow and later extended it to binary and ternary networks [2], [3]. Reviews of edge inference identify memory movement, limited on-chip resources, and software integration as system-level constraints rather than secondary implementation details [5].

Recent LUT-based approaches such as PolyLUT, NeuraLUT, and NeuraLUT-Assemble reduce inference cost by mapping increasingly complex learned functions into FPGA logic [6]–[8]. Their main contribution is a new model representation, whereas the present work keeps the model fixed and studies the surrounding execution path. A 2026 study introduced an hls4ml flow for radiation-hard Microchip devices and provides relevant evidence that SmartHLS and PolarFire are viable targets [9]. BeagleBoard has also documented an integration route for SmartHLS accelerators on BeagleV-Fire [13], and Microchip provides the SmartHLS/Libero tool chain used here [14].

The contribution of this report is therefore not a new neural architecture or a claim that the affine latency expression is itself novel. Instead, the H–K decomposition is used as an analysis lens for a controlled, platform-specific attribution of FPGA arithmetic, memory placement, host staging, and MMIO-lifecycle costs. The model and interface are held constant while one hardware or software factor is changed at a time.

III. Neural-Network Background and Experimental Model

A neural network is a sequence of parameterized transformations. A basic neuron forms a weighted sum, $z = \sum_i w_i x_i + b$, and applies a nonlinear activation such as the rectified linear unit, $\mathrm{ReLU}(z) = \max(0, z)$. Training learns the weights and biases from examples; inference keeps those parameters fixed and only evaluates the transformations. This report studies inference, not training.

In a multilayer perceptron (MLP), every neuron in a fully connected layer can use every value from the previous layer. Figure 2 shows the exact inference graph used here. A non-overlapping 2×2 mean reduction converts a 28×28 image into $14 \times 14 = 196$ unsigned 8-bit features. FC1 contains 32 hidden neurons, and each hidden neuron consumes all 196 features; thus, 196 and 32 denote different quantities rather than two coefficients of the same type. Signed 8-bit weights are accumulated in 32-bit integers, after which requantization and ReLU produce 32 unsigned 8-bit hidden activations. FC2 uses those 32 activations to compute 10 signed 32-bit class logits, and `argmax` selects the predicted digit.

The fixed model contains 6,634 parameters and performs 6,592 multiply–accumulate operations per image. It was selected as a controlled accelerator workload: it is large enough to expose arithmetic and data-movement costs, but small enough to permit bit-exact CPU/FPGA comparison for every logit.

IV. Platform and Workload

A. Hardware and Tool Flow

The target is BeagleV-Fire with a Microchip PolarFire SoC MPFS025T. Linux runs on the RISC-V processor complex, while the neural kernel is implemented in the FPGA fabric. SmartHLS translates the C/C++ inference kernel to RTL, and Libero SoC integrates, places, routes, and generates the device image. The CPU communicates with the accelerator through the board's CPU–FPGA control and shared-memory path. The

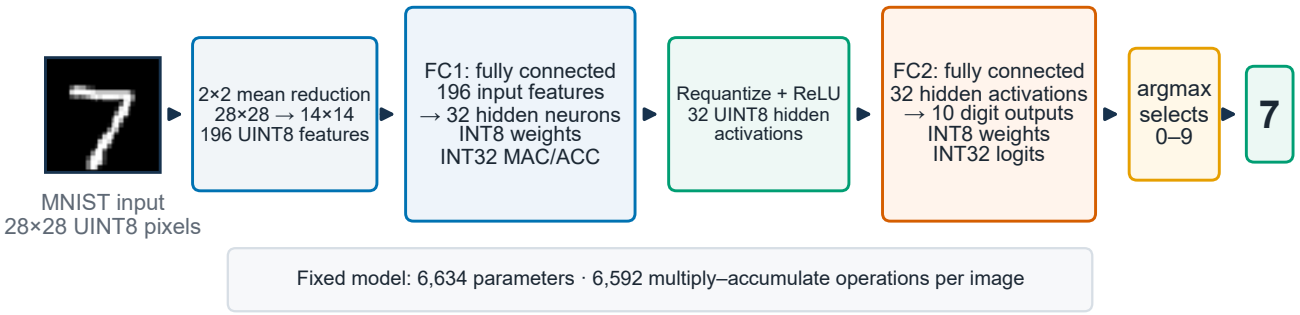


Fig. 2. Inference graph of the fixed INT8 MLP evaluated in this study. A 2×2 mean reduction produces 196 input features. Each of 32 FC1 hidden neurons consumes all 196 features; requantization and ReLU then produce 32 hidden activations. FC2 maps those activations to 10 INT32 digit logits, and `argmax` returns the predicted class.

target accelerator clock period is 7.5 ns, corresponding to approximately 133.33 MHz.

In the final optimized configuration, the complete execution path is:

1) the CPU reads an image from ordinary cacheable memory and reduces it into a 196-byte cacheable local buffer;
2) one contiguous `memcpy` stages the reduced input into a non-cacheable, accelerator-visible shared-DDR buffer;
3) the persistent driver supplies the input/output addresses, batch size, and start command through MMIO;
4) the FPGA, acting as an AXI initiator, reads the input buffer, computes the MLP, and writes 32-bit logits to the shared output buffer; and
5) the CPU reads the logits and selects the predicted class.

Consequently, the implementation is a CPU–FPGA collaborative pipeline, not "pure FPGA" inference.

Figure 1 maps these operations onto physical execution domains. The CPU owns image preparation, invocation setup through persistent MMIO, argmax, and correctness checking. Shared DDR contains a $B \times 196$-byte feature buffer and a $B \times 10$-word logit buffer; MMIO carries their addresses, the batch size, and control/status values. No separate DMA-engine copy is invoked: the FPGA accesses both buffers directly through its AXI initiator interface. Its batch controller advances through images sequentially, while four independent W1 banks allow four FC1 hidden neurons to be evaluated concurrently. This separation also explains the latency model used later: software and driver work contribute mainly to the fixed call cost H, whereas FPGA arithmetic and per-image transfer contribute mainly to K.

B. Model and Data

Each 28×28 MNIST image is reduced to 14×14 by averaging non-overlapping 2×2 regions, producing 196 input values. The classifier is a $196 \rightarrow 32 \rightarrow 10$ MLP containing 6,634 parameters and 6,592 multiply–accumulate operations per image. Inputs and hidden activations are unsigned 8-bit values, weights are signed 8-bit values, and biases and accumulators are signed 32-bit values.

All correctness and performance measurements use the complete canonical 10,000-image MNIST test set. The fixed model correctly classifies 9,428 images, or 94.28%. Because this work optimizes execution without retraining, every valid implementation must reproduce the same predictions and all 100,000 logits exactly. This bit-exact comparison is used as the correctness criterion throughout.

C. CPU Reference

The CPU reference is a single-threaded C++17 implementation of the same preprocessing, integer MLP, and argmax operations, compiled with `g++ -O3`. Each of five timed runs processes all 10,000 images already resident in memory using `std::chrono::steady_clock`. An untimed full-dataset validation pass precedes each timed run. Dataset loading and correctness checking are excluded from the timing boundary. The resulting mean is 109.393 μs/image (sample standard deviation 0.052 μs/image), equivalent to about 9,141 images/s.

FPGA full-pipeline time uses the matched in-memory boundary: it includes image reduction, buffer staging, accelerator interaction, and argmax, while retaining accelerator-call time as a separate component. CPU affinity, frequency-governor state, and compiler version were not archived, and the CPU runs were not interleaved with the final FPGA factorial experiment. CPU-relative speedups are therefore descriptive comparisons to this archived reference rather than claims of cross-system generality.

V. Experimental Method

A. Latency Decomposition

For one accelerator invocation that processes B images, the measured batch-level call time is

$$T_{\text{call}}(B) = H + BK, \tag{1}$$

where H is the fixed cost paid once per invocation and K is the incremental call cost of one additional image. The same host-observed interval can be decomposed as

$$T_{\text{call}}(B) = T_{\text{launch}} + T_{\text{service}}(B) + T_{\text{return}}. \tag{2}$$

Here, T_{launch} covers driver/MMIO preparation and the start command; T_{service} covers AXI/DDR access and FPGA execution

while the CPU waits or polls; and T_{return} covers completion detection and return to the caller. Accelerator-call latency is therefore a host-observed wall-clock interval, not pure FPGA-kernel time. AXI activity, FPGA execution, and CPU waiting overlap inside T_{service} and must not be added as independent wall-clock terms. The RTL co-simulation cycle count more closely approximates pure kernel time, although the board design did not include a fabric-side cycle counter.

The complete in-memory path is

$$T_{\text{full}}(B) = T_{\text{pre}}(B) + T_{\text{stage}}(B) + T_{\text{call}}(B) + T_{\text{post}}(B), \tag{3}$$

where T_{pre} covers the 2×2 mean reduction, T_{stage} is the contiguous copy into the shared feature buffer, and T_{post} covers logit access, `argmax`, and result bookkeeping. File I/O and batch-formation waiting are outside this boundary. Per-image quantities are obtained by division by the number of images in the call:

$$t_{\text{call}}(B) = \frac{T_{\text{call}}(B)}{B} = K + \frac{H}{B}, \qquad t_{\text{full}}(B) = \frac{T_{\text{full}}(B)}{B}. \tag{4}$$

This model gives distinguishable predictions. A better driver should reduce H without materially changing K, whereas wider accelerator parallelism should reduce K without eliminating host invocation cost. If both parameters change substantially, the experiment probably altered more than its intended factor.

B. Correctness, Timing, and Safety Gates

Each hardware version is checked by software simulation, RTL co-simulation, place-and-route timing, and full-dataset board testing before its performance is accepted. A configuration fails validation if any prediction or logit differs from the software reference, setup or hold slack is negative, or board temperature reaches 70°C.

The final comparison uses $B \in \{1, 2, 4, 8, 16, 32, 64, 128, 256\}$. Each configuration–batch cell contains 20 complete 10,000-image runs. Four replays of a fixed randomized nine-batch schedule produce 180 run-level observations per configuration, following one untimed warm-up for each batch size. If the final group contains fewer than B images, the program passes the true remaining count to the accelerator rather than padding or discarding images. The H–K fit therefore uses the observed effective batch size, defined as images divided by hardware calls.

Means, sample standard deviations, standard errors, and two-sided 95% Student-t confidence intervals are archived for each timing field. The affine model is fitted by ordinary least squares to the 180 run-level accelerator-call observations for each configuration. These intervals and fits describe within-session timing variation; they are not evidence of generalization across boards or boot sessions.

The engineering batch-knee criterion was fixed before the final 2×2 analysis as the smallest B for which the next doubling improves full-pipeline latency by both less than 2% and less than 1 μs/image. This identifies diminishing returns and does not require the curve to reverse direction.

TABLE I

Post-route FPGA resource use and timing closure. Resource counts are capacity costs rather than independent objectives. For S/H slack, ↑ denotes greater timing margin; bold marks the larger setup or hold value, and * marks the selected final hardware.

Configuration	4LUT	LE (%)	DSP	S/H (ns)↑
P0 (legacy weights)	21,435	96.11	5	+0.198/**+0.056**
P1 (LSRAM weights)	16,564	79.22	5	+1.227/+0.045
P2 (two lanes)	16,972	80.76	9	+1.179/+0.045
P4* (four lanes)	17,716	83.83	17	**+1.388**/+0.048

The trained parameters, quantization rules, and integer arithmetic are frozen before optimization. Every configuration is checked against both predicted classes and raw logits. Within each configuration, batch sizes are measured in a fixed randomized interleaving schedule to reduce temperature or background-load drift. P2 and P4 are necessarily measured in their respective hardware-deployment sessions rather than falsely treated as one cross-hardware randomized session.

C. Independent Factors and Version Labels

The experiments vary four conceptually independent factors. B is the number of images processed by one accelerator invocation. The hardware configurations are denoted P0, P1, P2, and P4. P0 is the legacy single-lane baseline, P1 is the single-lane LSRAM implementation, and P2/P4 contain two/four first-layer neuron lanes. P0 and P1 therefore have the same arithmetic parallelism: their comparison isolates weight-storage mapping, whereas P1–P2–P4 is the lane-scaling comparison.

The two software factors are denoted C and D. The accelerator-visible input and output buffers remain non-cacheable so that the FPGA observes shared-memory updates without explicit cache maintenance. C_0 is a controlled comparator in which the reduction loop produces 196 fine-grained stores directly in the non-cacheable input buffer. The final staging design, C_1, computes the complete reduction in a 196-byte cacheable local buffer and then performs one contiguous copy to the same accelerator-visible buffer. D_0 opens, maps, unmaps, and closes MMIO resources for every call; D_1 retains the mapping until process exit. The earlier hardware–batch isolation matrix fixes C_0, D_0 across all rows, whereas all final P2/P4–driver comparisons and final operating-point measurements fix C_1. The two matrices were collected in different sessions and batch ranges; their fitted intercepts are therefore not compared causally across sessions. This notation prevents batch size, neuron parallelism, host staging, and driver lifetime from being described as a single optimization step.

VI. Design Evolution

This section explains why each design change was introduced. Quantitative comparisons are consolidated in Section VII.

A. Batch Axis and Sequential Baseline

The initial sequential accelerator was bit-exact and required 167.908 μs/image for the archived board-level end-to-end path; accelerator call time was not separately archived. RTL co-simulation reported approximately 7,251 cycles, or 54.4 μs

TABLE II

HARDWARE AND BATCH-SIZE ISOLATION WITH DIRECT STAGING (C_0) AND TRANSIENT MMIO (D_0) FIXED. VALUES ARE MEAN μS/IMAGE; BOLD MARKS THE MINIMUM LATENCY IN EACH BATCH-SIZE COLUMN OF EACH PANEL.

Configuration	$B = 1$	$B = 2$	$B = 4$	$B = 8$	$B = 16$
(a) Full-pipeline latency ↓					
P0 (1 lane, legacy weights)	219.608	160.091	131.731	117.780	110.694
P1 (1 lane, LSRAM)	219.745	159.077	131.191	117.570	110.751
P2 (2 lanes)	191.188	136.365	107.443	92.266	85.337
P4 (4 lanes)	**183.388**	**124.805**	**96.288**	**80.635**	**72.938**
(b) Accelerator-call latency ↓					
P0 (1 lane, legacy weights)	171.641	115.114	88.516	75.414	68.597
P1 (1 lane, LSRAM)	172.068	113.903	87.933	75.307	68.625
P2 (2 lanes)	141.979	90.716	63.853	49.641	43.484
P4 (4 lanes)	**135.040**	**78.986**	**52.456**	**37.754**	**30.900**

at 133.33 MHz. The later batch-capable P0 implementation provided the first matched full-pipeline measurement: at $B = 1$, the full pipeline required 219.608 μs/image, or 2.007 times the CPU latency. Together, these observations showed that much of the measured time was outside the arithmetic kernel.

Batching amortized per-call work. Fitting the early measurements produced approximately $H = 107.878$ μs/call, consistent with an independently measured empty-call cost of 101.186 μs. At $B = 16$, accelerator-call time fell to 68.597 μs/image, but the full pipeline still required 110.694 μs/image. Batching thus exposed a fixed-cost bottleneck but did not remove its cause.

The initial batch experiment covered P0 over $B = 1$–16. A later P2 experiment extended the range to $B = 256$ without introducing another parallelism level. Both address the effect of batch size, but each is interpreted at its fixed hardware and software configuration.

B. Weight Storage: P0 to P1

The batch baseline consumed 96.11% of available logic elements (LEs), leaving insufficient headroom for a wider data path. Mapping weights into larger FPGA RAM blocks reduced LE use to 79.22% and improved setup slack, while latency remained nearly unchanged. This is an important negative result: storage remapping makes a later parallel design feasible but is not itself a performance optimization.

C. Neuron-Lane Parallelism: P1, P2, and P4

P1 evaluates the 32 FC1 hidden neurons sequentially. P2 introduces two concurrent neuron lanes and divides the FC1 weights into independent banks so that both lanes can read weights without a memory-port conflict. P4 extends the same organization to four lanes. Each pipeline accepts a new multiply–accumulate operation every cycle after filling. The model, quantization, FC2 implementation, interface, clock target, and batch semantics remain unchanged; the intervention therefore targets FC1 arithmetic parallelism rather than image-level batch parallelism. Place-and-route results are summarized in Table I, and latency comparisons are reported in Table II.

D. Cacheable Host Staging

The accelerator-visible buffers remain non-cacheable so that the FPGA observes shared-memory updates without explicit cache-maintenance operations. The inefficiency lies not in this requirement itself, but in producing all 196 reduced pixels through fine-grained uncached stores inside the preprocessing loop. Such stores cannot use a cacheable local working set and interleave image arithmetic with shared-memory writes. The final host path therefore computes the complete reduction in a 196-byte cacheable local array and issues one contiguous `memcpy` to the non-cacheable input buffer. A controlled P2–D_0 ablation retains the FPGA image, model, quantization, driver, batch interface, and timing protocol; its measured effect is reported in Fig. 3(a). Cacheable staging is then fixed for all final-system comparisons.

E. MMIO Lifetime: D_0 to D_1

The generated driver originally opened, mapped, unmapped, and closed its MMIO resources for every accelerator invocation. The persistent implementation retains the mapping until process exit while leaving the accelerator hardware and data representation unchanged. This intervention is intended to reduce fixed per-call overhead rather than per-image arithmetic cost; its measured effect on H and K is reported in Fig. 3(b)–(c).

Table I reports four-input lookup tables (4LUTs), logic elements (LEs), dedicated digital-signal-processing (DSP) blocks, and S/H setup/hold timing slack. Across Tables I–III, an arrow is used only when a quantitative metric has an unambiguous preferred direction under fixed experimental conditions. Controlled variables such as B, and FPGA resource counts that express implementation cost or capacity, carry no arrow. Bold type marks a columnwise measured optimum only where such a direction is defined; a superscript asterisk identifies a selected design or operating point instead of claiming that every entry in its row is optimal.

VII. RESULTS

The results answer three questions in sequence: whether weight remapping directly improves latency, how neuron-lane parallelism interacts with batch size under a fixed host path, and how host staging and persistent MMIO change the final hardware configurations.

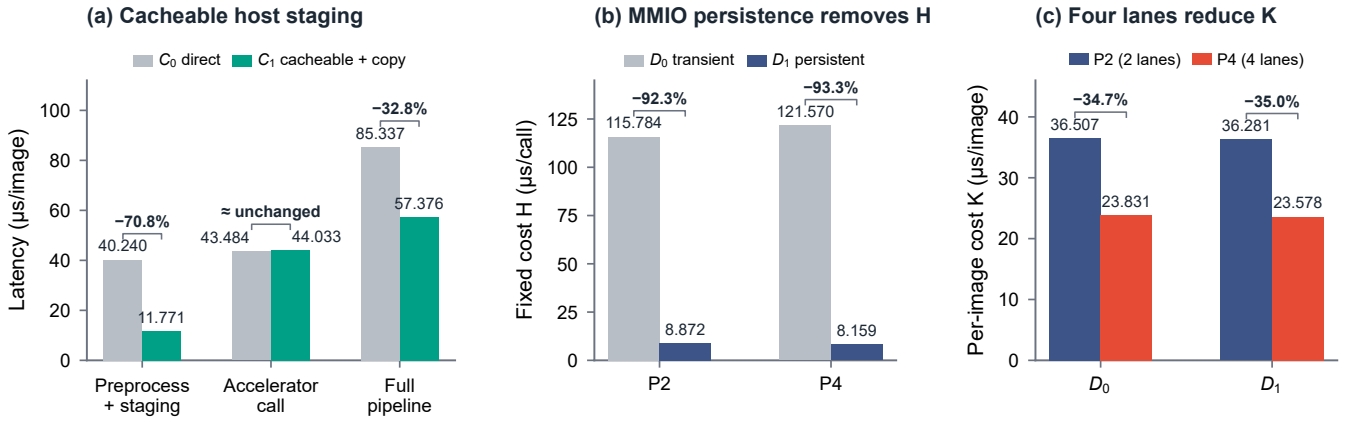


Fig. 3. Controlled software and hardware ablations; shorter bars are better in all panels. (a) Replacing fine-grained direct stores (C_0) with cacheable staging and one contiguous copy (C_1) reduces preprocessing plus staging by 70.75% and full-pipeline latency by 32.77%, while accelerator-call latency is essentially unchanged. (b) Within both P2 and P4 hardware, persistent MMIO (D_1) reduces the fitted fixed invocation cost H by 92.3–93.3% relative to transient MMIO (D_0). (c) Under both driver lifetimes, increasing FC1 parallelism from P2 to P4 reduces the fitted incremental cost K by 34.7–35.0%.

A. Hardware and Batch Effects under a Fixed Host Path

P0 and P1 have nearly identical latency at every batch size, confirming that LSRAM remapping released physical resources but did not itself accelerate the one-lane computation. The P1 call time at $B = 4$, 87.933 μs/image, is below the 109.393 μs/image CPU reference, but this is not a full-pipeline speedup: the corresponding full pipeline still requires 131.191 μs/image.

The first observed full-pipeline mean below the CPU reference occurs for P2 at $B = 4$: 107.443 μs/image, or 1.018× the CPU rate. Because the margin is only 1.8%, it is reported as a mean crossover rather than a robust break-even claim. P2 at $B = 8$ and $B = 16$ provides clearer margins of 1.186× and 1.282×, respectively. P4 further reduces the full-pipeline latency under this fixed hardware-isolation host path to 72.938 μs/image at $B = 16$, a 1.500× CPU speedup.

This matrix deliberately retains one common direct-staging, transient-MMIO host path to isolate the hardware and batch axes; it is not the final host design. After the staging ablation selected C_1, all final P2/P4–driver comparisons and operating-point measurements fixed cacheable staging. Keeping the two experiments separate prevents CPU preprocessing and driver lifetime from being mixed with hardware configuration and batch size.

B. Host Staging Ablation

Figure 3(a) isolates host data preparation without changing the FPGA image or MMIO lifecycle. Cacheable staging reduces preprocessing time by 70.75% and full-pipeline latency by 32.77%, while accelerator-call latency changes little. The result supports treating staging as a host-path optimization rather than an FPGA-kernel optimization.

C. Hardware Parallelism and Driver Lifetime

Figure 3(b)–(c) separates the two optimization effects. Increasing FPGA parallelism from P2 to P4 reduces K by 34.7% with the transient driver and 35.0% with the persistent driver. In contrast, persistent mapping reduces P4's H from 121.570 to 8.159 μs/call, a 93.3% reduction, while having little effect on its incremental cost. Here, lower H and K are unambiguously preferable; a lower knee B only means that near-saturated efficiency is reached with fewer simultaneously available images. All four fits had $R^2 \geq 0.999977$ over the measured batch range, supporting the affine model used for this comparison.

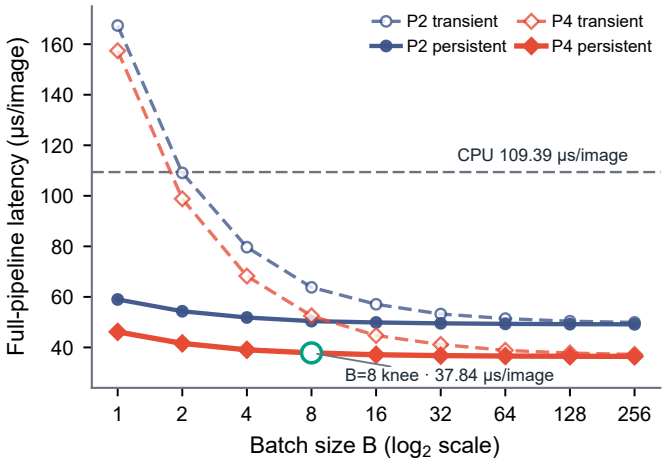


Fig. 4. Full-pipeline latency versus batch size for the matched P2/P4 × transient/persistent-MMIO experiment with cacheable host staging fixed. Blue and red identify P2 and P4; dashed open-marker curves and solid filled-marker curves identify transient and persistent MMIO, respectively. The gray dashed line is the CPU reference; the green-ringed $B = 8$ point is the first batch satisfying the predefined knee criterion.

Figure 4 visualizes the matched full-pipeline comparison across batch sizes.

D. Final P4 Operating Point

Table III reports the measured operating points for the final P4 configuration with cacheable staging and persistent MMIO.

TABLE III

KNEE IDENTIFICATION FOR THE FINAL P4–C_1D_1 SYSTEM. THROUGHPUT RETENTION IS NORMALIZED TO THE MAXIMUM MEASURED THROUGHPUT AT $B = 256$. GAIN TO $2B$ IS THE REDUCTION IN FULL-PIPELINE LATENCY AFTER THE NEXT DOUBLING. THE KNEE TEST PASSES WHEN THAT GAIN IS BOTH BELOW 1 μS/IMAGE AND BELOW 2%; * MARKS THE SMALLEST PASSING BATCH. BOLD MARKS ONLY COLUMNWISE MEASURED EXTREMA.

B	t_{full} (μs/img)↓	Maximum-throughput retention (%)↑	Gain to $2B$ (μs/img; %)	Knee test
1	46.153	79.00	4.548; 9.85	–
2	41.605	87.64	2.556; 6.14	–
4	39.049	93.38	1.210; 3.10	–
8*	37.839	96.36	0.736; 1.94	*First pass*
16	37.103	98.28	0.338; 0.91	Pass
32	36.765	99.18	0.171; 0.47	Pass
64	36.594	99.64	0.100; 0.27	Pass
128	36.494	99.92	0.030; 0.08	Pass
256	**36.464**	**100.00**	–	–

For P4 persistent, the improvement from $B = 4$ to $B = 8$ is 3.10%, or 1.210 μs/image. The next doubling improves latency by only 1.94%, or 0.736 μs/image, so $B = 8$ is the first point satisfying the predefined knee criterion. It achieves 37.839 μs/image, approximately 26,428 images/s, with 96.36% of the maximum measured throughput while requiring only one thirty-second as many simultaneously available images as $B = 256$. Its 95% confidence interval for full latency is [37.812, 37.865] μs/image. Its descriptive speedup relative to the archived CPU reference is 2.891×.

The maximum measured throughput occurs at $B = 256$: 36.464 μs/image, or approximately 27,425 images/s, which is 3.000× the CPU rate. Relative to $B = 8$, it reduces per-image latency by only 3.63% (equivalently, throughput increases by 3.77%). The $B = 128$ to 256 improvement is only 0.083%. Thus, $B = 256$ is the numerical throughput extreme, whereas $B = 8$ is the smallest measured batch that provides near-saturated compute efficiency under the stated knee criterion.

All four final configurations retain 94.28% accuracy, zero mismatched images, and zero mismatched logits relative to the CPU reference. The highest measured P4 temperature is 60.125°C, below the 70°C stopping threshold.

VIII. Discussion

The initial result demonstrates the difference between a fast accelerator kernel and a fast application. Although RTL co-simulation indicated a much shorter arithmetic time, the full invocation crossed the CPU–FPGA boundary and paid preprocessing, shared-memory, and driver costs. Batching amortized part of the fixed cost but could not remove its source.

The interventions affected different parts of the system. Moving weights to LSRAM was an enabling resource optimization: it reduced logic occupancy and made wider hardware feasible, but did not materially accelerate the one-lane design. Increasing FC1 parallelism from P2 to P4 primarily reduced the incremental accelerator-call cost K. Persistent MMIO primarily reduced the fixed call cost H, while cacheable host staging reduced time outside the accelerator call by replacing fine-grained uncached stores with local cacheable computation followed by one contiguous copy. The final result therefore required the combination of resource mapping, FPGA parallelism, host data preparation, and driver-lifetime optimization.

Per-image latency continued to decrease through $B = 256$, so the measured curve does not contain a reversal that would define a mathematical optimum. Instead, $B = 8$ is the smallest measured batch satisfying the predefined diminishing-return criterion: it retains 96.36% of the maximum measured throughput, and doubling to $B = 16$ improves latency by only 0.736 μs/image (1.94%). It is therefore a selected compute-efficiency knee, not a mathematical optimum. Batch-formation or queueing delay was not measured, so this operating point should not be interpreted as a universal optimum for online request latency. More generally, the experiment shows that large batches can hide fixed overhead, whereas persistent driver state removes much of that overhead and permits efficient operation at smaller batches.

IX. Limitations and Future Work

The study uses one board, one small MLP, one primary boot session, in-memory input data, and no external power meter. The result supports a claim about this platform and workload, not all networks or FPGA systems. The CPU reference has five low-variance repetitions, but CPU affinity, frequency-governor state, and compiler version were not archived and it was not interleaved with the final FPGA experiment. CPU-relative speedups should therefore be interpreted as descriptive comparisons to that reference. The within-session confidence intervals likewise do not quantify variation across reboots or boards.

Accuracy is fixed by the trained model, so this work does not claim improved classification quality. The timing boundary also excludes file I/O, live sensor input, batch-formation delay, and queueing. Temperature was monitored for safety, but temperature alone cannot establish energy efficiency.

Future work should repeat the full factorial experiment across board restarts and, if possible, multiple boards; measure input power with an external meter; add a fabric-side cycle counter to separate accelerator and interconnect time directly; and evaluate larger MLPs or LUT-based networks with greater arithmetic intensity. Recent NeuraLUT-Assemble and PolarFire-oriented hls4ml work provide two candidate paths [8], [9]. The same H–K method can then test whether persistent drivers and small-batch operation generalize beyond this MLP.

X. Conclusion

This study examined an in-memory RISC-V–FPGA MNIST inference path on BeagleV-Fire rather than evaluating the FPGA kernel in isolation. Weight mapping released the resources required for wider hardware, four-lane FC1 parallelism reduced incremental accelerator-call cost, persistent MMIO reduced fixed invocation cost, and cacheable staging reduced CPU preprocessing overhead. Their combination reached 37.839 μs/image at the $B = 8$ compute-efficiency knee and retained 96.36% of the maximum measured throughput. The numerical throughput extreme at $B = 256$ reached 36.464 μs/image; the two points correspond to descriptive speedups of 2.891× and approximately 3.00× over the archived CPU reference. On this platform and workload, useful application-level acceleration required joint

optimization of FPGA arithmetic, host data preparation, and driver lifetime. Whether the same cost balance holds for larger networks and other FPGA SoCs remains to be evaluated.

References

[1] J. Duarte *et al.*, "Fast inference of deep neural networks in FPGAs for particle physics," *J. Instrum.*, vol. 13, no. 7, p. P07027, 2018, doi: 10.1088/1748-0221/13/07/P07027.

[2] F. Fahim *et al.*, "hls4ml: An open-source codesign workflow to empower scientific low-power machine learning devices," in *TinyML Research Symposium*, 2021, arXiv:2103.05579.

[3] J. Ngadiuba *et al.*, "Compressing deep neural networks on FPGAs to binary and ternary precision with hls4ml," *Mach. Learn.: Sci. Technol.*, vol. 2, no. 1, p. 015001, 2020, doi: 10.1088/2632-2153/aba042.

[4] Y. Umuroglu *et al.*, "FINN: A framework for fast, scalable binarized neural network inference," in *Proc. ACM/SIGDA Int. Symp. FPGA*, 2017, pp. 65–74, doi: 10.1145/3020078.3021744.

[5] M. M. H. Shuvo, S. K. Islam, J. Cheng, and B. I. Morshed, "Efficient acceleration of deep learning inference on resource-constrained edge devices: A review," *Proc. IEEE*, vol. 111, no. 1, pp. 42–91, 2023, doi: 10.1109/JPROC.2022.3226481.

[6] M. Andronic and G. A. Constantinides, "PolyLUT: Learning piecewise polynomials for ultra-low latency FPGA LUT-based inference," in *Proc. ICFPT*, 2023, pp. 60–68, doi: 10.1109/ICFPT59805.2023.00012.

[7] M. Andronic and G. A. Constantinides, "NeuraLUT: Hiding neural network density in Boolean synthesizable functions," in *Proc. FPL*, 2024, pp. 140–148, doi: 10.1109/FPL64840.2024.00028.

[8] M. Andronic and G. A. Constantinides, "NeuraLUT-Assemble: Hardware-aware assembling of sub-neural networks for efficient LUT inference," arXiv:2504.00592, 2025.

[9] E. A. Smith, "Enabling low-latency machine learning on radiation-hard FPGAs with hls4ml," arXiv:2602.15751, 2026.

[10] Y. LeCun, C. Cortes, and C. J. Burges, "The MNIST database of handwritten digits." [Online]. Available: http://yann.lecun.com/exdb/mnist/

[11] BeagleBoard.org Foundation, "BeagleV-Fire introduction." [Online]. Available: https://docs.beagleboard.org/boards/beaglev/fire/01-introduction.html

[12] Microchip Technology Inc., "PolarFire SoC FPGAs." [Online]. Available: https://www.microchip.com/en-us/products/fpgas-and-plds/system-on-chip-fpgas/polarfire-soc-fpgas

[13] BeagleBoard.org Foundation, "BeagleV-Fire meets SmartHLS: Unlock FPGA acceleration," Oct. 29, 2025. [Online]. Available: https://www.beagleboard.org/blog/2025-10-29-beaglev-fire-meets-smarthls-unlock-fpga-acceleration

[14] Microchip Technology Inc., "SmartHLS compiler." [Online]. Available: https://www.microchip.com/en-us/products/fpgas-and-plds/fpga-and-soc-design-tools/smarthls-compiler

Appendix A
Software Environment and Toolchain

The FPGA tool flow ran on a 64-bit Windows 11 host, whereas all reported CPU and CPU–FPGA measurements ran under Linux on BeagleV-Fire. Table IV records the components needed to reproduce the hardware build and board measurements. Exact Python-package versions and the standalone CPU compiler build were not retained; this provenance limitation is reported in the main text. This table contains descriptive provenance rather than optimization metrics, so no preferred-direction arrows apply.

Appendix B
Reproducible Build and Deployment Workflow

A. Frozen Inputs and Independent References

Before hardware generation, the model topology, integer arithmetic, weights, preprocessing, test-image order, and expected logits were frozen. The four canonical MNIST gzip files matched their canonical MD5 values and decompressed byte-for-byte to the IDX files used by the programs. IDX headers confirmed 60,000 training and 10,000 test images of size 28×28 with matching label counts. No exact image duplicates were found within either split or across the training and test splits. All reported performance tests used the complete 10,000-image test set; training data were not timed.

The reference path first evaluated the frozen INT8 model on the host and then on the RISC-V CPU. Generated P2 and P4 weight-bank headers were checked against the original unsplit W_1 array for every mapping. The comparison implementation continued to read the unsplit array so that a bank-generation error could not be shared silently by the reference and accelerator. Before board deployment, deterministic tests compared all ten logits per image rather than only the final class.

B. Hardware Build Gates

Each hardware revision passed the following ordered gates:

1) SmartHLS software testing established bit-exact agreement with the independent integer reference.
2) RTL co-simulation exercised every supported batch size and checked every output logit. Pipeline reports were inspected for the intended initiation interval and independent weight-bank reads.
3) Libero integrated the accelerator into the PolarFire SoC reference design and completed synthesis, place-and-route, and timing analysis at a 7.5 ns requested clock period.
4) Resource reports were checked for 4LUTs, logic elements, flip-flops, DSP blocks, RAM1K20 blocks, and RAM64x12 blocks. A design was not flashed if setup or hold slack was negative, resource use exceeded its predefined limit, or the generated-artifact manifest failed its hash check.

The LSRAM experiment used three levels of evidence rather than relying on an HLS estimate alone: SmartHLS's memory report, generated RTL memory attributes, and the physical Libero initialization report. This established that fixed weights had moved into RAM1K20 blocks. It did not imply that the neural network had been "stored instead of computed"; multiply–accumulate, requantization, ReLU, and control remained logic operations in the FPGA fabric.

C. Board Staging, Flashing, and Rollback

Before persistent flashing, the active gateware, device-tree overlay (DTBO), and full flash backup were retained as rollback material. The candidate bitstream, DTBO, and host executable were staged over SSH/SCP into a new board directory, and their SHA-256 values were compared with the host copies. Staging was rejected if a hash differed, the shared non-cacheable DDR device `/dev/udmabuf-ddr-nc-lb0` was absent, or temperature was at least 70°C.

The board updater then wrote the matched bitstream and DTBO and rebooted Linux. After reconnection, the active firmware hashes were compared with the staged files and the shared-buffer device was checked again. Only then were a smoke test, the complete 10,000-image correctness test, and the timed runs executed. A clean `sync` and Linux shutdown followed result retrieval and checkpoint verification. A preliminary sine-kernel deployment served only to verify this SmartHLS–Libero–shared-memory path; its approximately 10× internal result is not an MNIST result and is excluded from all main-text comparisons.

D. Host-Software Source-Transform Safeguard

SmartHLS compiles a transformed copy of the application under its generated `.hls/sw` directory. Therefore, changing project source and invoking only the final software-compile target can reuse stale transformed source. The formal P4 host-staging executables explicitly re-ran `soc_source_transform`; the source actually passed to the compiler was then checked for the cacheable downsampling routine, the contiguous copy to the shared buffer, and all staging calls. Driver sources, executable hashes, target logs, and RTL hashes were also audited. An earlier P4 software build failed this audit because it contained the unstaged path; it was retained as provenance but excluded from the formal 2×2 comparison. The corrected software build did not regenerate hardware, and its RTL tree was verified byte-identical to the already deployed P4 design.

Appendix C
Controlled Configuration Definitions

The experiments separate hardware organization, host staging, driver lifetime, and batch size. Table V defines the notation used throughout the paper. P0–P4 identify FPGA hardware configurations, C_0/C_1 identify host-data paths, D_0/D_1 identify MMIO lifetimes, and B is the number of images submitted per accelerator call. These entries define controlled factors rather than universally ranked alternatives, so preferred-direction arrows do not apply.

Two negative controls informed the sequence. Explicitly adding another `-O3` flag changed P2 $B = 16$ full-pipeline

TABLE IV
CORE SOFTWARE ENVIRONMENT USED FOR HARDWARE GENERATION AND BOARD-LEVEL MEASUREMENT.

Component	Version or configuration	Role in this study
Libero SoC Design Suite	2026.1.0.17	Integrated the accelerator with the PolarFire SoC reference design, performed place-and-route and timing analysis, reported physical resources, and generated the deployable FPGA image.
SmartHLS Compiler	2026.1	Translated the C/C++ MLP kernel to register-transfer-level (RTL) hardware, generated the Linux host interface, ran software tests and RTL co-simulation, and cross-compiled the accelerator host application.
RTL simulation and synthesis	QuestaSim Pro ME 2025.3; Synplify Pro Y-2026.03M, build 112R	Executed RTL co-simulation and synthesized the accelerator within the Libero flow.
PolarFire SoC build support	MSS Configurator 2026.1; SoftConsole RISC-V toolchain 2022.2-RISC-V-747; Device Tree Compiler 1.5.0	Generated the microprocessor subsystem (MSS), boot-support components, and device-tree overlay. The SoftConsole IDE was not used for the Linux inference application.
Board Linux environment	Debian 13.6 (riscv64), Linux 6.6.75 Microchip build	Executed the CPU reference and CPU–FPGA host programs; Secure Shell/Secure Copy (SSH/SCP) supported command execution, deployment, and result retrieval.
Application and analysis	C++17 with `-O3`; Python with NumPy	Implemented the CPU reference, prepared and checked model parameters, generated weight banks, and parsed timing results.

TABLE V
CONTROLLED HARDWARE AND SOFTWARE FACTORS USED IN THE EXPERIMENTS.

Factor or configuration	Controlled change	Experimental interpretation
P0	One FC1 neuron lane with the legacy weight-storage organization.	Hardware baseline for the configuration–batch matrix.
P1	One FC1 lane with fixed W_1/W_2 weights mapped to LSRAM/RAM1K20.	Isolates weight placement; releases logic resources without changing arithmetic parallelism.
P2	Two FC1 neuron lanes supplied by two independent W_1 banks.	Tests whether neuron-level parallelism reduces the incremental per-image cost K.
P4	Four FC1 neuron lanes supplied by four independent W_1 banks.	Final hardware configuration and the upper parallelism level evaluated.
$C_0 \rightarrow C_1$	Replaces fine-grained direct stores to non-cacheable shared memory with cacheable local reduction followed by one 196-byte `memcpy` per image.	Isolates CPU preprocessing and buffer-staging cost without changing the FPGA image.
$D_0 \rightarrow D_1$	Replaces per-call MMIO open/map/unmap/close operations with a mapping retained until process exit.	Isolates fixed driver-lifecycle overhead without changing accelerator arithmetic.
B	Number of images submitted in one accelerator call; images within a batch are processed sequentially.	Amortizes fixed invocation cost H and exposes the latency–throughput trade-off without introducing image-level FPGA parallelism.

time by only 0.56that SmartHLS already optimized the RISC-V host compilation. Separately, LSRAM mapping substantially reduced logic use but left the one-lane cycle count and board latency nearly unchanged. These results redirected effort toward cacheable staging, MMIO lifetime, and true lane parallelism rather than compiler-flag or storage-only explanations.

P2/P4 parallelism is within one image—two or four hidden neurons are evaluated concurrently—whereas the images in a batch remain sequential. The FPGA accesses the shared feature and logit buffers directly through AXI; no separately timed DMA-engine copy follows the host `memcpy`.

APPENDIX D
VALIDATION, MEASUREMENT, AND ARTIFACT PROTOCOL

A. Correctness and Timing Boundaries

Correctness testing covered all 10,000 predictions and all 100,000 INT32 logits. Every retained configuration reproduced 9,428 correct classifications (94.28%), zero mismatched images, zero mismatched logits, and an invariant prediction checksum. The stable accuracy across batch sizes is consequently a property of one deterministic model and test set, not evidence that different subsets were evaluated.

Four conceptual timing regions were kept distinct. CPU preprocessing covers the 28×28 to 14×14 mean reduction in cacheable memory, and host staging covers the subsequent copy into the shared feature buffer. The archived timers report these two regions together as "preprocessing" because they were not instrumented separately. Accelerator-call time begins with control-register setup and includes start, real AXI/DDR access, FPGA execution, polling, and return. Postprocessing covers logit access, `argmax`, and checksum/accuracy bookkeeping. Full-pipeline latency is the sum of these in-memory regions; file I/O, model training, batch-formation delay, and queueing are excluded. RTL co-simulation cycles are reported separately and are never presented as physical on-board cycles.

The standalone CPU reference used the same preprocessing, integer arithmetic, weights, and `argmax`. Each of five timed runs processed the complete in-memory test set after an untimed validation pass. Early P0–P2 batch-matrix cells used one warm-up and five fixed-randomized full-dataset runs. The extended P2 and final P4 experiments used $B \in \{1, 2, 4, 8, 16, 32, 64, 128, 256\}$, one untimed warm-up per batch size, and four separately logged replays of a fixed randomized batch schedule. Each replay contained five runs per batch, giving 20 complete 10,000-image runs per configuration–batch cell and 180 runs per configuration. Randomization occurred among batch sizes within a configuration; the P2 and P4 hardware were necessarily measured in their respective deployment sessions.

A final partial batch used its actual remaining count rather than padding or dropping images. For configured B =

32, 64, 128, 256, the final call contains 16 images and the effective batch sizes are approximately 31.949, 63.694, 126.582, and 250.000, respectively. Fits use $B_{\text{eff}} = 10000/(\text{hardware calls})$ rather than assuming that every call is full.

B. Analysis and Stop Rules

Call totals were fitted to $T_{\text{call}}(B) = H + BK$ using effective batch size. The practical knee was fixed before final analysis as the smallest B for which the next doubling improved full-pipeline latency by both less than 2% and less than 1 μs/image. Means, sample standard deviations, standard errors, and two-sided 95% Student-t intervals were retained for run-level measurements; the reported H–K intervals are conventional within-session OLS intervals.

An experiment stopped before performance interpretation if any logit differed, place-and-route timing was negative, a required hash failed, the shared-buffer interface was absent, the board disconnected, or temperature reached 70°C. For P4, logic-element use also had to remain at or below 90% and $B = 256$ RTL cycles had to decrease by at least 25% relative to P2 before flashing. The highest temperature observed in final P4 experiments was 60.125°C. These checks establish operational safety and run integrity; they do not substitute for external power measurement.

C. Retained Reproducibility Artifacts

The checkpoints retain HLS and host sources, generated weight-bank manifests, transformed compile sources, generated transient and persistent drivers, bitstream and DTBO, host executables, software-test and co-simulation logs, place-and-route resource/timing reports, raw board logs, parsed JSON statistics, temperature records, dataset audit, randomized schedules, decision log, and SHA-256 manifests. Checkpoint manifests were re-verified with zero hash errors before the board was shut down. The locked source revisions include gateware builder commit `2aa1c93612da`, FPGA HLS libraries commit `721c4dd484a3`, and HSS commit `4bea7e24f3e5`.

Exact compiler and Python versions for the final CPU/model runs, CPU affinity, and frequency-governor state were not captured. This missing provenance is why CPU-relative speedups are described as comparisons with an archived reference rather than universal platform ratios.